

Experimental Methods in Computer Science

Experiment Design

André Miguel Correia Cardoso 2022222265

Luis Miguel Ferreira Vaz 2022214707

Pedro Costa 2022220304

University of Coimbra, Mestrado em Engenharia Informática 2025/2026

Contents

1	Introduction	4
2	Bug Finding	4
2.1	Experimental Design	4
2.1.1	Research Question	4
2.1.2	Independent Variables	5
2.1.3	Dependent Variables	5
2.1.4	Controlled Variables	5
3	Code Generation	6
3.1	Context and Motivation	6
3.1.1	Dataset and Problem Selection	6
3.1.2	Prompting and Generation Procedure	6
3.1.3	Evaluation Methodology	6
3.1.4	Performance Metrics	6
3.2	Experimental Design	7
3.2.1	Research Questions	7
3.2.2	Variables	7
3.2.3	Independent Variables	7
3.2.4	Controlled Variables	8
3.3	Results and Correctness Analysis	9
3.4	Execution Time Analysis	9
3.4.1	Descriptive Statistics	10
3.4.2	Distribution by Difficulty	10
3.4.3	Per-Problem Comparison	11
3.4.4	Summary	11
3.5	Hypothesis 1: Correctness Comparison	11
3.5.1	(i) Hypothesis	11
3.5.2	(ii) Statistical Test Selection	12
3.5.3	(iii) Data Characteristics	12

3.5.4	(iv) Test Computation	12
3.5.5	(v) Results	13
3.5.6	(vi) Decision and Conclusion	13
3.6	Hypothesis 2: Execution Time Comparison	13
3.6.1	(i) Hypothesis	13
3.6.2	(ii) Statistical Test Selection	13
3.6.3	(iii) Data Characteristics	14
3.6.4	(iv) Test Computation	14
3.6.5	(v) Results	15
3.6.6	(vi) Decision and Conclusion	15
3.7	Hypothesis 3: Impact of Problem Difficulty	15
3.7.1	(i) Hypothesis	15
3.7.2	(ii) Statistical Test Selection	15
3.7.3	(iii) Data Characteristics	16
3.7.4	(iv) Test Computation	16
3.7.5	(v) Results	16
3.7.6	(vi) Decision and Conclusion	16

1 Introduction

Large Language Models (LLMs) are increasingly being integrated into software development workflow. As their use becomes more widespread, it is important to evaluate the extent to which developers can rely on these tools in practical software engineering contexts.

This report was developed as part of the Experimental Methods in Computer Science curricular unit and addresses the following central problem statement:

Can developers rely on LLM-based tools to support software development activities?

Given the wide scope of this question, the experiment focuses on two specific areas of software development: Code Generation and Code Review (more specifically Bug Finding). These two areas were studied separately, using distinct datasets and statistical analysis methods suited to each task.

Despite these differences, both parts of the experiment share a common setting: they are based on the Python programming language and evaluate the same LLM models. This allows for a structured comparison of model performance across two relevant software development activities, contributing to a broader understanding of the capabilities and limitations of LLM-based tools.

2 Bug Finding

Code Review, and more specifically Bug Finding, refers to the practice of providing an LLM with source code and asking it to inspect the code for potential defects. Depending on the experimental setup, this process may involve supplying only the relevant code files, or it may include additional contextual information such as failing tests, execution outputs, error messages, or brief descriptions of the expected behaviour.

This experiment focuses on the bug detection capabilities of the selected LLMs. In particular, the models are tasked with analysing Python source code and identifying the locations of real bugs. The objective is to evaluate how effectively each model can support developers in detecting defects during code review activities.

2.1 Experimental Design

2.1.1 Research Question

The research question for this experiment is:

How does the choice of Large Language Model affect bug-finding performance in real-world Python bugs?

2.1.2 Independent Variables

The independent variables are:

- **LLM Model (2 levels):**
- **Language (1 level) :**
 - Python

Even though there is only one language being used in the bug, it is still considered an independent variable.

2.1.3 Dependent Variables

The dependent variables used in the statistical comparison are:

- **Recall:** Fraction of the real bugs identified.

$$Recall = \frac{TP}{TP + FN}$$

- **F1-Score:** F1-score combines recall and precision into a single metric that penalises both missed bugs and incorrectly reported bugs:

$$F1 = \frac{2TP}{2TP + FP + FN}$$

Precision was excluded, since, due to its formula, when an LLM reports zero bugs, it will generate a division by 0. Since F1-score is the harmonic mean between Recall and Precision we decided to opt for that dependent variable.

2.1.4 Controlled Variables

In order to reduce unnecessary variables and isolate each LLM's performance, we kept the following variable constant:

- **Scenarios:** Both LLMs are fed the same bug scenarios
- **Prompt:** All prompts follow the same format. They just vary from scenario to scenario in the Source Files section, since they include that specific scenario's buggy code.
- **New chat for each submission:** In order to avoid outputs based on previous context, each submission is made in a new chat.

3 Code Generation

3.1 Context and Motivation

This experiment evaluated the reliability of Large Language Models (LLMs) in generating SQL queries. SQL was selected because it provides a standardized environment in which generated solutions can be automatically executed and validated against predefined test cases.

3.1.1 Dataset and Problem Selection

A dataset consisting of 45 SQL programming problems was collected from the LeetCode platform. The selected problems were evenly distributed across three difficulty levels (Easy, Medium, and Hard), enabling the evaluation of model performance under varying levels of complexity.

3.1.2 Prompting and Generation Procedure

For each problem, both LLMs received the same prompt template and were instructed to generate a single SQL query as the solution. Each problem was submitted independently, and no interaction history was shared between problems.

Each model was allowed a single attempt per problem. This design choice was made to evaluate first-response reliability and to avoid bias introduced by iterative refinement or multiple attempts.

3.1.3 Evaluation Methodology

The generated solutions were submitted to the LeetCode evaluation platform, which automatically executed each query against a set of hidden test cases. The platform determined whether each solution was accepted or rejected based on its execution results.

The evaluation was based on the acceptance status, which ensures that semantically equivalent queries producing identical results are considered correct.

3.1.4 Performance Metrics

In addition to correctness, execution time reported by the platform was also recorded. This metric is used to analyze the query’s efficiency, enabling a comparative analysis of computational performance between models.

Although execution time is influenced by factors such as query structure and database optimization strategies, it provides a practical indicator of relative efficiency between generated solutions.

3.2 Experimental Design

To ensure experimental consistency, all problems were presented using the same prompt structure. Only the problem statement and input/output specifications varied between tasks.

Furthermore, each problem was submitted in an independent conversation context, preventing information from previous interactions from influencing subsequent responses. As a result, every generated solution constitutes an independent observation.

3.2.1 Research Questions

The experiment was designed to investigate the reliability of Large Language Models (LLMs) in SQL code generation. Specifically, the study aims to address the following research questions:

- **RQ1:** Is there a statistically significant difference in the correctness of SQL queries generated by ChatGPT Mini and Gemini Flash?
- **RQ2:** Does problem difficulty significantly influence the correctness of SQL queries generated by Large Language Models?

3.2.2 Variables

The experimental design includes two independent variables and multiple dependent variables used to evaluate the reliability and performance of Large Language Models (LLMs) in SQL code generation.

3.2.3 Independent Variables

IV1: LLM Model

This variable represents the Large Language Model used to generate SQL queries. It has two levels:

- ChatGPT Mini
- Gemini Flash

IV2: Problem Difficulty

This variable represents the difficulty level of the SQL programming problems selected from the dataset. It has three levels:

- Easy
- Medium

- Hard

DV1: Query Outcome Classification (Primary Outcome)

This variable classifies the outcome of each generated SQL query based on its execution and correctness. It is defined as a categorical variable with the following levels:

- **Correct:** the query executes successfully and passes all hidden test cases
- **Incorrect Logic:** the query executes successfully but fails one or more hidden test cases due to logical or constraint violations
- **Syntax or Execution Error:** the query fails to execute due to syntax errors or runtime issues

This single dependent variable captures both execution success and semantic correctness, allowing a clear and non-overlapping classification of model output quality.

DV2: Execution Time (Secondary Performance Metric)

This variable records the execution time reported by the evaluation platform for each query. It is treated as a continuous variable and used as a proxy for query efficiency. It is not used as a measure of correctness, but instead as a secondary performance indicator to compare computational efficiency across models.

3.2.4 Controlled Variables

Several factors that could influence the quality of the generated solutions were kept constant throughout the experiment to ensure that observed differences could be attributed to the independent variables under investigation.

CV1: Prompt Template

All problems were presented using the same prompt template. Only the problem statement and the associated input/output specifications varied between tasks.

CV2: Number of Generation Attempts

Each model was allowed a single attempt per problem. No retries, prompt refinements, or iterative interactions were permitted.

CV3: Evaluation Platform

All generated solutions were evaluated using the LeetCode platform under the same evaluation procedure and hidden test cases.

CV4: Conversation Context

Each problem was submitted in an independent conversation context. This ensured that information from previous interactions could not influence subsequent responses.

3.3 Results and Correctness Analysis

The generated SQL solutions were evaluated on 61 problems per model. Because each problem was submitted once and the platform returned a binary outcome (Accepted or Not Accepted), correctness and accuracy are equivalent in this setting and are computed as:

$$\text{Accuracy} = \frac{\text{Accepted Solutions}}{\text{Total Problems}}$$

Table 1 summarises the overall correctness of both models.

Model	Accepted	Total	Accuracy
ChatGPT	59	61	96.72%
Gemini	58	61	95.08%

Table 1: Overall correctness of the SQL code generation models.

The two models performed very similarly overall, with ChatGPT obtaining a slightly higher acceptance rate. The difference is driven by one additional rejected solution in Gemini’s output. In both cases, the models achieved perfect performance on Easy problems and near-perfect performance on Medium problems.

Table 2 breaks down the results by problem difficulty.

Difficulty	ChatGPT	Gemini	Total Problems
Easy	21/21 (100.00%)	21/21 (100.00%)	21
Medium	21/21 (100.00%)	20/21 (95.24%)	21
Hard	17/19 (89.47%)	17/19 (89.47%)	19

Table 2: Correctness by difficulty level.

The hardest problems produced the lowest acceptance rate for both models, indicating that problem complexity had a clear effect on performance. The Hard category accounts for both of the ChatGPT failures and for two of the three Gemini failures. One additional Gemini failure occurred in the Medium category.

3.4 Execution Time Analysis

Beyond correctness, the execution time reported by the LeetCode platform was recorded for each accepted submission and used as a secondary performance indicator. The following analysis is based exclusively on accepted solutions, as rejected queries do not produce meaningful runtime measurements.

3.4.1 Descriptive Statistics

Table 3 summarises the central tendency and dispersion of execution times across all accepted submissions.

Model	Mean (ms)	Median (ms)	Std. Dev. (ms)	Accepted
ChatGPT	109	82	122	57/60
Gemini	107	82	95	56/60

Table 3: Descriptive statistics of execution time for accepted submissions.

The two models present nearly identical median runtimes (82 ms for both), and their means differ by only 2 ms. However, both distributions exhibit considerable right skew, driven by a small number of outliers. Notably, ChatGPT produced a 764 ms result on Problem 1158 (Medium) and Gemini produced results of 644 ms and 445 ms on Problems 577 and 1084 (Easy), respectively. These outliers inflate the mean substantially relative to the median, which is why the median is a more representative measure of central tendency for this dataset.

3.4.2 Distribution by Difficulty

Figure 1 shows the mean execution time grouped by difficulty level and model. ChatGPT presents a higher average on Hard problems, largely due to the 229 ms result on Problem 1890, while Gemini achieves a lower average on that category. Conversely, Gemini’s Easy average is inflated by the two outliers mentioned above.

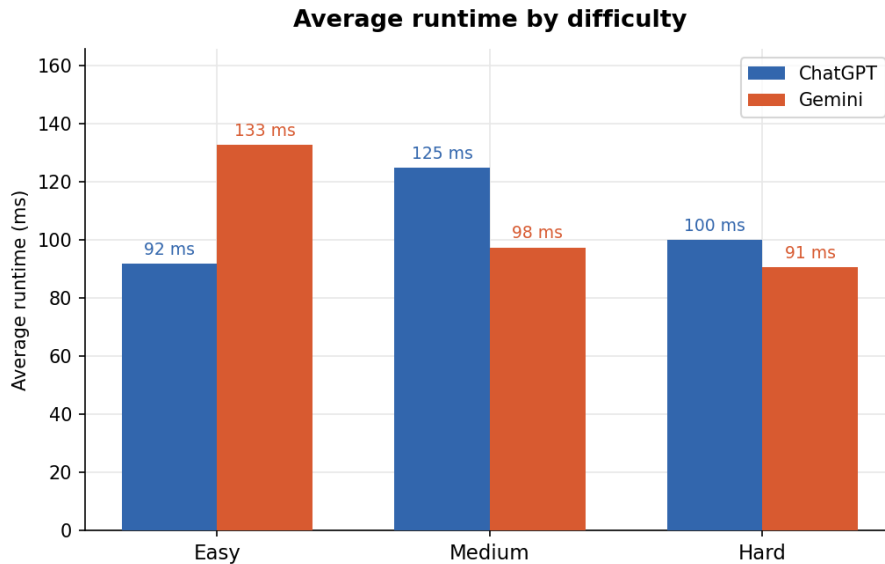


Figure 1: Mean execution time by difficulty level.

The box plots in Figure 2 corroborate this observation. Both models display similar interquartile ranges across difficulty levels, indicating that the majority of queries execute within a comparable

and consistent time range. The outliers visible in the Easy category for Gemini are the primary source of distributional asymmetry for that model.

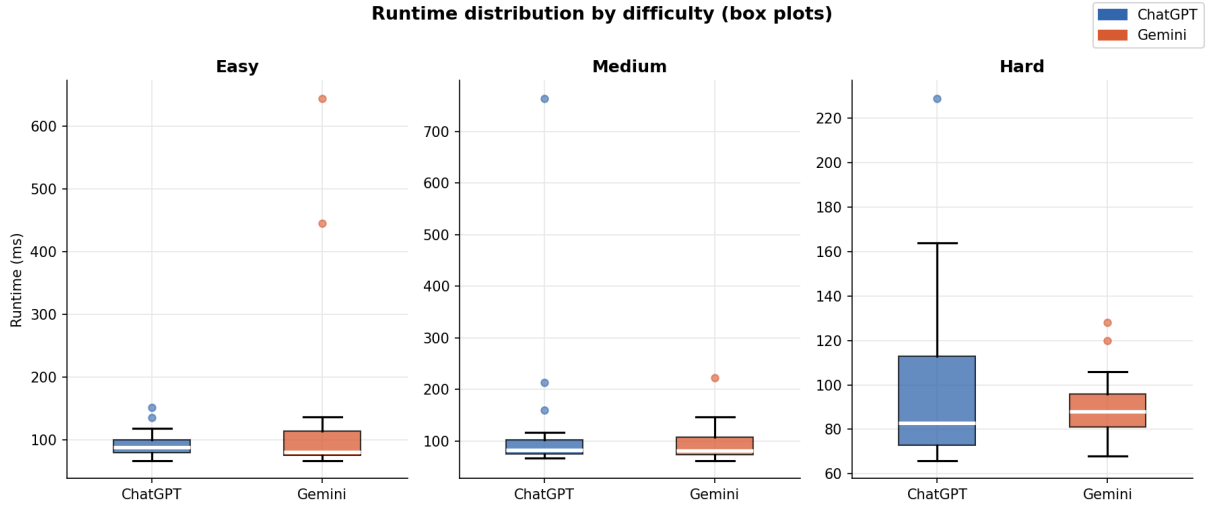


Figure 2: Runtime distribution by difficulty level (box plots). Boxes show the interquartile range; horizontal lines indicate the median; whiskers extend to the minimum and maximum non-outlier values.

3.4.3 Per-Problem Comparison

Figure 3 provides a direct head-to-head comparison of execution times for each problem accepted by both models. No consistent dominance is observed for either model: each outperforms the other on roughly half the shared problems. The largest discrepancies arise from the individual outliers previously identified and are not representative of a systematic performance gap.

3.4.4 Summary

Overall, the execution time analysis indicates that ChatGPT and Gemini produce SQL queries of comparable runtime efficiency. Both models concentrate their runtimes within a narrow band (60–150 ms) across all difficulty levels. The minor differences observed in aggregate statistics are attributable to isolated outliers rather than to a systematic performance advantage of either model. Consequently, execution time does not constitute a meaningful differentiator between the two models under the conditions of this experiment.

3.5 Hypothesis 1: Correctness Comparison

3.5.1 (i) Hypothesis

Null Hypothesis (H_0):

There is no statistically significant difference in the correctness of SQL queries generated by ChatGPT Mini and Gemini Flash.

Alternative Hypothesis (H_1):

There is a statistically significant difference in the correctness of SQL queries generated by ChatGPT Mini and Gemini Flash.

3.5.2 (ii) Statistical Test Selection

Since both models were evaluated on the same set of problems and the outcome is binary (Accepted or Not Accepted), the paired comparison is appropriately evaluated using McNemar’s test.

Data Assumption Checks:

- **Paired nominal data:** Met. Each problem acts as its own control, producing a binary outcome for both ChatGPT and Gemini.
- **Discordant pairs:** Given the high accuracy of both models, the number of discordant pairs (where one model is correct and the other is incorrect) is expected to be small. Therefore, the exact binomial version of McNemar’s test is used instead of the asymptotic chi-square approximation.

3.5.3 (iii) Data Characteristics

For each problem, the paired results can be classified into one of four mutually exclusive cases, forming a 2×2 contingency table.

	Gemini Correct	Gemini Incorrect
ChatGPT Correct	56	3
ChatGPT Incorrect	2	0

Table 4: Contingency table for correctness outcomes.

The off-diagonal cells represent the discordant pairs: 3 problems were solved by ChatGPT but failed by Gemini, and 2 problems were solved by Gemini but failed by ChatGPT.

3.5.4 (iv) Test Computation

The exact McNemar’s test evaluates whether the discordant pairs are equally distributed using a binomial distribution. Under H_0 , the probability of a discordant pair falling into either off-diagonal cell is 0.5.

The test statistic is the number of discordances of one type:

$$S = \min(\text{ChatGPT Only}, \text{Gemini Only}) = \min(3, 2) = 2$$

The exact two-sided p-value is calculated from the binomial distribution $\mathcal{B}(n = 3 + 2, p = 0.5)$.

3.5.5 (v) Results

Test Statistic: $S = 2.0$

p-value (Exact): $p = 1.0000$

Significance Level (α): 0.05

3.5.6 (vi) Decision and Conclusion

- Since $p = 1.0000 > \alpha = 0.05$: **Fail to reject H_0**

Conclusion: There is no statistically significant difference in the correctness of SQL queries generated by ChatGPT Mini and Gemini Flash ($p = 1.0000$, exact McNemar’s test). The observed difference (one additional accepted problem for ChatGPT) is negligible and easily attributable to random variation rather than a consistent performance advantage. Both models demonstrated highly comparable and reliable performance on this dataset.

3.6 Hypothesis 2: Execution Time Comparison

The third hypothesis concerns the execution time of accepted SQL queries and is formulated as follows:

3.6.1 (i) Hypothesis

Null Hypothesis (H_0):

There is no statistically significant difference in the execution times of SQL queries generated by ChatGPT Mini and Gemini Flash.

Alternative Hypothesis (H_1):

There is a statistically significant difference in the execution times of SQL queries generated by ChatGPT Mini and Gemini Flash.

The comparison is based exclusively on problems accepted by both models, resulting in a paired sample of $n = 56$ observations.

3.6.2 (ii) Statistical Test Selection

Since each problem was submitted to both models and the same evaluation platform was used, the observations are naturally paired. The choice of statistical test depends on the distributional properties of the paired differences $d_i = \text{ChatGPT}_i - \text{Gemini}_i$.

Data Assumption Checks:

- **Normality assessment:** The Shapiro-Wilk test was applied to the paired differences.
- **Visual inspection:** A Q-Q plot was generated to complement the formal normality test.

3.6.3 (iii) Data Characteristics

Normality Analysis:

The Shapiro-Wilk test yielded $W = 0.5337$ with $p < 0.0001$, providing strong evidence against the assumption of normality. The null hypothesis of the Shapiro-Wilk test (that the differences follow a normal distribution) is therefore rejected at $\alpha = 0.05$.

Test	Statistic	p-value
Shapiro-Wilk	$W = 0.5337$	$p < 0.0001$

Table 5: Shapiro-Wilk normality test on paired runtime differences.

The Q-Q plot in Figure 4 confirms this finding. The sample quantiles deviate substantially from the theoretical normal quantiles, particularly in the tails, indicating heavy-tailed and non-normal behaviour in the distribution of paired differences.

Selected Test: Given that the normality assumption is violated, the **Wilcoxon signed-rank test** was selected. This non-parametric test is appropriate for paired samples and does not require the differences to be normally distributed.

3.6.4 (iv) Test Computation

The Wilcoxon signed-rank test ranks the absolute values of the paired differences and computes the test statistic W as the smaller of the sum of positive ranks and the sum of negative ranks:

$$W = \min(W^+, W^-)$$

where W^+ is the sum of ranks associated with positive differences and W^- is the sum of ranks associated with negative differences.

For the observed data:

- Number of paired observations: $n = 56$
- Mean difference: $\bar{d} = -1.04$ ms
- Median difference: $\tilde{d} = 1.00$ ms

3.6.5 (v) Results

Test Statistic: $W = 796.0$

p-value: $p = 0.9870$

Significance Level (α): 0.05

Effect Size: Cohen's $d = -0.0082$ (negligible)

3.6.6 (vi) Decision and Conclusion

- Since $p = 0.9870 > \alpha = 0.05$: **Fail to reject H_0**

Conclusion: There is no statistically significant difference in the execution times of SQL queries generated by ChatGPT Mini and Gemini Flash ($W = 796.0$, $p = 0.9870$). The effect size (Cohen’s $d = -0.0082$) is negligible, confirming that the two models produce queries of comparable runtime efficiency. This result is consistent with the descriptive analysis presented in the previous section, where both models exhibited nearly identical median runtimes and largely overlapping distributions.

3.7 Hypothesis 3: Impact of Problem Difficulty

3.7.1 (i) Hypothesis

This hypothesis addresses whether the difficulty level of the problem significantly influences the ability of Large Language Models to generate correct SQL queries.

Null Hypothesis (H_0):

The proportion of correct SQL queries generated by the models is independent of the problem difficulty.

Alternative Hypothesis (H_1):

The proportion of correct SQL queries generated by the models depends significantly on the problem difficulty.

3.7.2 (ii) Statistical Test Selection

To answer this research question, the results from both models were combined to test the general effect of difficulty on LLM performance. The data forms a categorical contingency table relating difficulty (Easy, Medium, Hard) to correctness (Accepted, Not Accepted).

Data Assumption Checks:

- **Expected Frequencies:** Because the models achieved near-perfect performance on Easy and Medium problems, the number of incorrect outcomes for these categories is extremely low.
- **Test Adjustment:** To address the sparse data, the difficulty levels were collapsed into two categories: "Hard" and "Not Hard" (combining Easy and Medium). This allows the use of **Fisher’s Exact Test**, which provides an exact p-value for 2×2 contingency tables with small cell counts.

3.7.3 (iii) Data Characteristics

Combining the 61 problems evaluated across both models yields $n = 122$ total observations. The collapsed contingency table is as follows:

	Correct	Incorrect	Total
Not Hard (Easy + Medium)	83	1	84
Hard	34	4	38
Total	117	5	122

Table 6: Collapsed contingency table for correctness by difficulty.

The table shows that only 1 out of 84 submissions failed for Not Hard problems, whereas 4 out of 38 submissions failed for Hard problems.

3.7.4 (iv) Test Computation

Fisher’s Exact Test calculates the exact probability of observing the given contingency table, or any table more extreme, assuming that the null hypothesis (independence) is true.

The two-sided p-value is computed from the hypergeometric distribution of the cell counts.

3.7.5 (v) Results

p-value (Fisher’s Exact Test): $p = 0.0323$

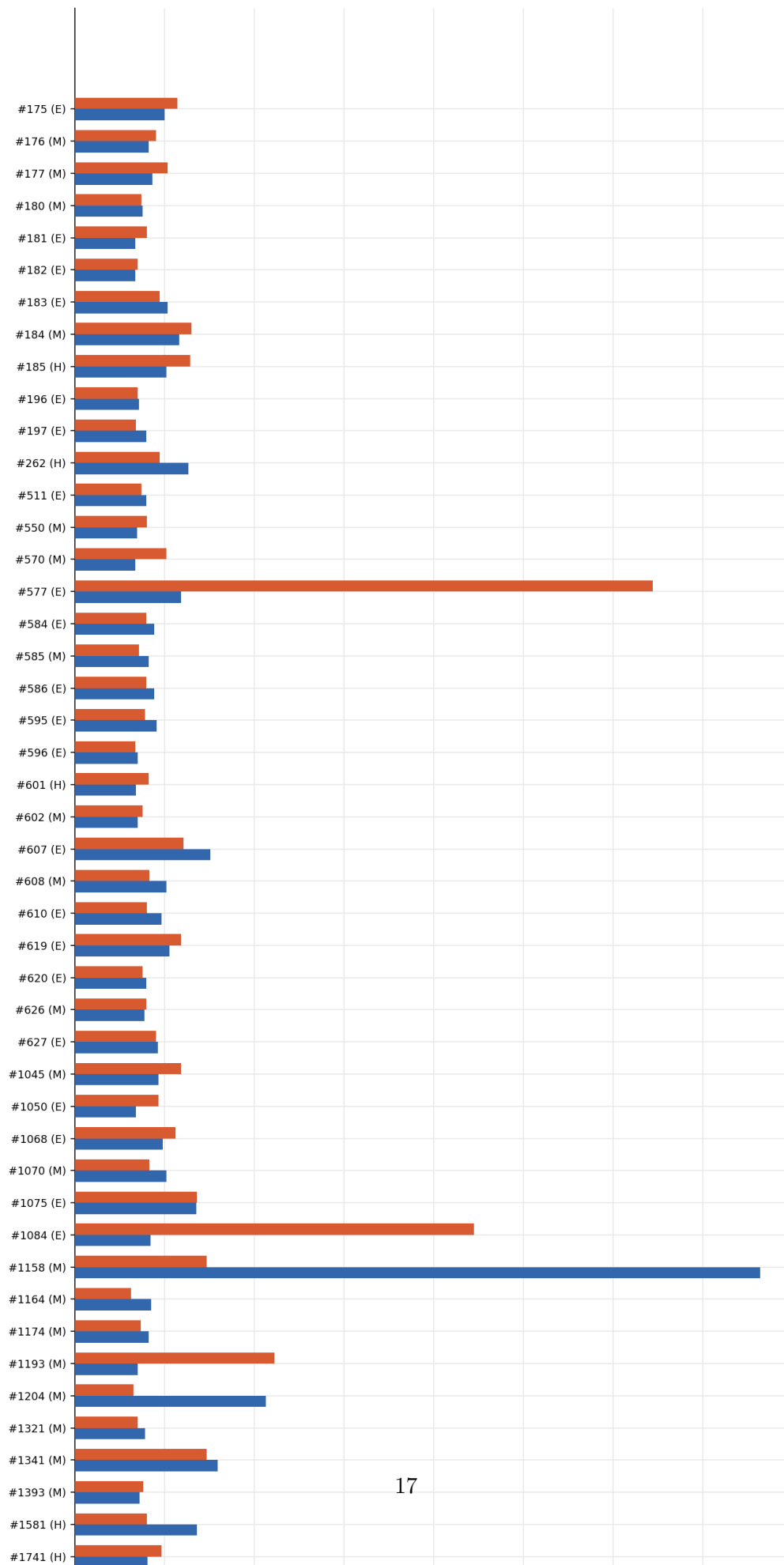
Significance Level (α): 0.05

3.7.6 (vi) Decision and Conclusion

- Since $p = 0.0323 \leq \alpha = 0.05$: **Reject** H_0

Conclusion: Problem difficulty has a statistically significant influence on the correctness of SQL queries generated by the models ($p = 0.0323$, Fisher’s Exact Test). Specifically, the models are significantly more likely to produce incorrect solutions when faced with Hard problems compared to Easy and Medium problems. This confirms that while LLMs are highly reliable for basic and intermediate SQL generation, their performance meaningfully degrades as problem complexity increases.

**Per-problem runtime — head-to-head
(problems accepted by both models)**



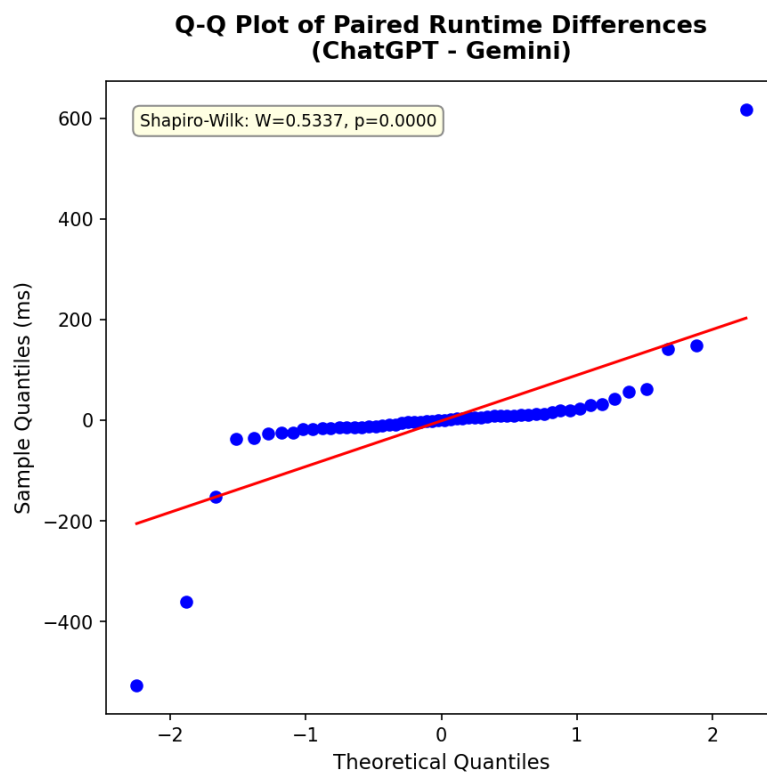


Figure 4: Q-Q plot of paired runtime differences (ChatGPT – Gemini). Deviations from the diagonal indicate non-normality.